

Database Laboratory Work 1 – Amankozha Altynbek

Relation A: Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1) A superkey is any attribute or set of attributes that uniquely identifies a tuple. Since EmpID, SSN, and Email are each individually unique, any set containing at least one of them is a superkey. Examples:

1. {EmpID}
2. {SSN}
3. {Email}
4. {EmpID, SSN}
5. {EmpID, Name}
6. {EmpID, SSN, Email, Phone, Name, Department, Salary} (the full relation, trivially a superkey)

2) A candidate key is a minimal superkey (no attribute can be removed without losing uniqueness). Based on the data and real-world semantics:

{EmpID} - a system-generated unique identifier.

{SSN} - unique per person by definition.

{Email} - assumed unique per company policy (one mailbox per employee).

Phone, Name, and Department are not candidate keys: names can repeat, department is shared by many employees, and phone numbers are not guaranteed unique.

3) EmpID is the best choice for the primary key.

reasons:

-It is a short, stable, system-generated surrogate value that never changes over an employee's tenure.

-SSN is sensitive personal data; using it as a primary key exposes it in every foreign key reference and log, which is a privacy/security risk.

-Email can change (e.g., after a name change or company rebranding), which would force cascading updates to every foreign key referencing it.

Therefore SSN and Email remain candidate keys (and should be enforced with a UNIQUE constraint), but EmpID is designated the primary key.

4) Phone is not listed as a candidate key, and nothing in the schema or sample data guarantees uniqueness. In practice, employees can share a landline/extension happens to be distinct, but that is just a property of this particular data instance, not a schema constraint uniqueness of an attribute can never be proven from a single sample; it must be guaranteed by a key constraint.

Relation B: Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1) Primary key - (StudentID, CourseCode, Section, Semester, Year)

2)

StudentID - distinguishes which student the registration record belongs to; without it, records for different students taking the same section would collide.

CourseCode - distinguishes which course is being registered for.

Section - the business rule states a student cannot register for the same course section in the same semester, implying different sections of the same course (in the same semester) are separate, valid registrations, so Section must be part of the key.

Semester and Year - the rule "a student can take the same course in different semesters" means (StudentID, CourseCode, Section) alone is not unique across time; Semester and Year together identify which offering the registration refers to.

No smaller subset is sufficient: dropping any one of these five attributes would allow two legitimate registrations to collide on the remaining attributes.

3) None are evident from the given attributes. Grade and Credits are dependent, descriptive attributes (Grade is assigned per registration; Credits depends on CourseCode/Section, not the other way around), so they cannot serve as part of a key. Unless the schema includes a surrogate RegistrationID (not given here), {StudentID, CourseCode, Section, Semester, Year} is the only candidate key.

Task 1.2 Foreign key

Given tables: Student, Professor, Course, Department, Enrollment.

Table (child)	Foreign Key Column	References (parent)	Meaning
Student	AdvisorID	Professor(ProfID)	Each student is advised by one professor
Course	DepartmentCode	Department(DeptCode)	Each course belongs to one department
Professor	Department	Department(DeptCode)	Each professor works in one department
Department	ChairID	Professor(ProfID)	Each department is chaired by one professor
Enrollment	StudentID	Student(StudentID)	Each enrollment record belongs to one student
Enrollment	CourseID	Course(CourseID)	Each enrollment record refers to one course

Note: Department.ChairID -> Professor.ProfID and Professor.Department -> Department.DeptCode form a mutual reference cycle. This is valid in practice but requires one of the two foreign keys to be made nullable/deferred (or added after both tables are populated), since a department cannot have a chair before at least one professor exists in it.

PART 2

Task 2.1 Hospital Management System

Entity	Type	Reason
Patient	Strong	Has its own unique identifier (Patient ID)
Doctor	Strong	Has its own unique identifier (Doctor ID)
Department	Strong	Has its own unique identifier (Department Code)
Appointment	Strong (associative/regular entity)	Has its own natural key (e.g., date/time + patient + doctor) and independent existence once scheduled; often modeled as a regular entity because it carries its own attributes
Prescription	Strong (associative entity)	Identified by its own ID, though it exists only in the context of a Patient–Doctor–Medication relationship
Room	Weak	Room numbers are only unique within a department (room 101 in Cardiology ≠ room 101 in Neurology); it depends on Department for its identity

Room is the clear weak entity: its partial key is Room Number, and its full identifying key is (DeptCode, RoomNumber), borrowing the owner Department's key — exactly the scenario the lab's "Common Mistakes" list warns about (weak entities need the owner's key included in their composite key).

2. Attributes per entity

Entity	Attribute	Classification
Patient	PatientID	Simple, key

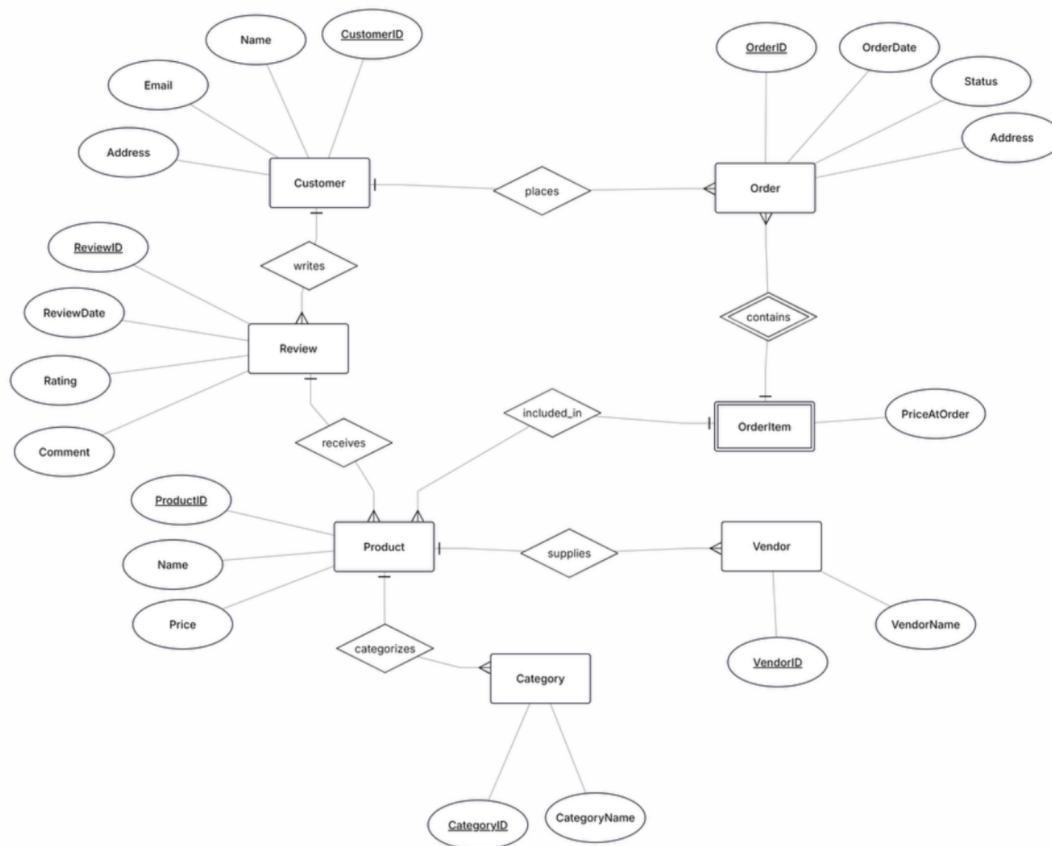
Entity	Attribute	Classification
Patient	Name	Simple (or composite: First/Last)
Patient	Birthdate	Simple
Patient	Address (Street, City, State, Zip)	Composite
Patient	Phone Numbers	Multi-valued
Patient	Insurance Info	Composite (Provider, Policy No., Group No.)
Patient	Age	Derived (from Birthdate)
Doctor	DoctorID	Simple, key
Doctor	Name	Simple
Doctor	Specializations	Multi-valued
Doctor	Phone Number	Simple
Doctor	Office Location	Simple
Department	DeptCode	Simple, key
Department	Name	Simple
Department	Location	Simple
Appointment	AppointmentID	Simple, key
Appointment	Date/Time	Simple
Appointment	Purpose	Simple
Appointment	Notes	Simple
Prescription	PrescriptionID	Simple, key
Prescription	Medication	Simple
Prescription	Dosage	Simple
Prescription	Instructions	Simple
Room	RoomNumber	Simple, partial key

3. Relationships and cardinalities

Relationship	Participants	Cardinality	Participation
Has	Department – Doctor	1:N	Total on Doctor, partial on Department
Located In	Department – Room	1:N (identifying)	Total on Room (weak entity)
Schedules	Patient – Doctor (via Appointment)	M:N (Patient : Doctor through Appointment)	Partial on both
Attends	Appointment – Room	N:1	Optional on Room
Writes	Doctor – Prescription	1:N	Total on Prescription, partial on Doctor
Issued To	Patient – Prescription	1:N	Total on Prescription, partial on Patient

Because Appointment links exactly one Patient and one Doctor but a Patient can have many Appointments (with many different Doctors) and vice versa, the Patient–Doctor relationship is effectively M:N, resolved through the Appointment associative entity, which itself carries the Date/Time, Purpose, and Notes attributes.

4-5. ER diagram and primary keys



- Strong entities: single-outline rectangles — Patient, Doctor, Department.
- Weak entity: double-outline rectangle — Room, connected to Department via a double-outline (identifying) diamond "Located In".
- Composite attributes: an oval attribute connected to sub-ovals (Address → Street, City, State, Zip).
- Multi-valued attributes: double-outline ovals — Patient Phone Numbers, Doctor Specializations.
- Derived attribute: dashed oval — Age.
- Primary keys underlined: PatientID, DoctorID, DeptCode, AppointmentID, PrescriptionID; Room's key is the composite (DeptCode, RoomNumber), underlined with DeptCode shown as a dashed underline (borrowed key).
- Cardinalities marked on each relationship diamond's connecting lines as 1, N, or M per the table above, with total participation shown as double lines (Room–Department, Prescription–Doctor, Prescription–Patient).

Task 2.2: E-commerce Platform

Entity	Key attributes
Customer	CustomerID (PK), Name, Email, BillingAddress
Order	OrderID (PK), OrderDate, Status, CustomerID (FK)

Entity	Key attributes
OrderItem	OrderID (FK, partial key), ProductID (FK, partial key), Quantity, PriceAtOrder
Product	ProductID (PK), Name, Description, CategoryID (FK), VendorID (FK)
Category	CategoryID (PK), CategoryName
Vendor	VendorID (PK), VendorName, ContactInfo
Review	ReviewID (PK), Rating, Comment, CustomerID (FK), ProductID (FK)
Inventory	ProductID (PK/FK), QuantityOnHand, WarehouseLocation
ShippingAddress	AddressID (PK), CustomerID (FK), Street, City, State, Zip

Relationships

- Customer (1) - places - (N) Order
- Order (1) - contains - (N) OrderItem; Product (1) - appears in - (N) OrderItem -> Order M:N Product resolved by OrderItem
- Category (1) - classifies - (N) Product
- Vendor (1) - supplies - (N) Product
- Customer (1) - writes - (N) Review; Product (1) - receives - (N) Review -> Customer M:N Product resolved by Review
- Product (1) - tracked in - (1) Inventory
- Customer (1) - has - (N) ShippingAddress

2. Weak entity

OrderItem is the weak entity. It has no independent existence or natural identifier of its own - a line item is meaningless without knowing which Order it belongs to. Its identifying (partial) key is ProductID, and its full key is the composite (OrderID, ProductID), where OrderID is the owner Order's key. It is connected to Order through a total, identifying relationship (an OrderItem cannot exist if its Order is deleted).

3)

Order-Product is M:N: one order can include many products, and one product can appear in many orders. This relationship needs its own attributes — Quantity and PriceAtOrder (the price at the time of purchase, which must be captured because the product's current listed price may change later) - which is exactly why it is resolved into the associative entity OrderItem rather than a plain M:N line.

A second M:N with attributes is Customer-Product via Review, which needs Rating, Comment, and ReviewDate attached to each specific review instance.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional dependencies

StudentID → StudentName, StudentMajor

ProjectID → ProjectTitle, ProjectType, SupervisorID

SupervisorID → SupervisorName, SupervisorDept

StudentID, ProjectID → Role, HoursWorked, StartDate, EndDate

(The last FD holds because Role, HoursWorked, StartDate, and EndDate describe one specific student's participation in one specific project — a fact that only makes sense per student-project pair.)

2. Problems

Redundancy:

- StudentName and StudentMajor repeat in every row for a student working on multiple projects.
- ProjectTitle, ProjectType, and SupervisorID (plus SupervisorName, SupervisorDept transitively) repeat in every row for every student assigned to the same project.

Anomalies:

- Update anomaly: If a supervisor changes department, every row containing that SupervisorID must be updated; missing even one row leaves the data inconsistent.
- Insert anomaly: A new project cannot be recorded until at least one student is assigned to it, because ProjectID/ProjectTitle/SupervisorID have no home outside a student-row. Likewise, a new supervisor cannot be added before they supervise a project with an assigned student.
- Delete anomaly: If the single student assigned to a project drops out and that row is deleted, all information about the project (title, type, supervisor) is lost along with the student's data.

3. 1NF

The table as given has no repeating groups or multi-valued attributes in a single cell, so it already satisfies 1NF (assuming each row represents one student–project–role combination, one value per cell). No violation to fix here.

4. 2NF

Primary key: {StudentID, ProjectID} (a student can work on several projects, and each project can involve several students, but each student's participation on a given project is a single row).

Partial dependencies (attributes depending on only part of the composite key):

- StudentID → StudentName, StudentMajor (depends only on StudentID, not on the full key)
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept (depends only on ProjectID)

2NF decomposition:

```
Student(StudentID, StudentName, StudentMajor)
```

```
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)
```

```
Participation(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)
```

Now every non-key attribute in each table depends on the whole key of that table - 2NF is satisfied.

5. 3NF

Transitive dependency: In the Project table, ProjectID → SupervisorID → SupervisorName, SupervisorDept. SupervisorName and SupervisorDept depend on SupervisorID, which is not a key of Project — a transitive dependency through a non-key attribute.

Final 3NF decomposition:

```
Student(StudentID, StudentName, StudentMajor)
```

```
Supervisor(SupervisorID, SupervisorName, SupervisorDept)
```

```
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID*)
```

```
Participation(StudentID*, ProjectID*, Role, HoursWorked, StartDate, EndDate)
```

(* denotes a foreign key.) Every non-key attribute in every table now depends only on the whole key of its own table and on nothing else — the schema is in 3NF (and, since each table's only remaining determinant is a candidate/primary key, also in BCNF).

Task 4.2: Advanced Normalization

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary key

A course section is uniquely identified by (TimeSlot, Room), since rooms are unique campus-wide and each room can only host one section at a given time slot — equivalently it is identified by (CourseID, InstructorID, TimeSlot), since each course section is taught by one instructor at one time. A student enrolls in multiple sections, so the row itself (one student in one section) needs the student too. The primary key of the whole table is therefore:

Primary Key = {StudentID, TimeSlot, Room}

This is the "trick": TimeSlot and Room alone determine CourseID, InstructorID, InstructorName, and Building (because a room+time slot pins down exactly one section), so CourseID is not needed in the key even though it looks like the natural per-course identifier.

2. Functional dependencies

StudentID → StudentMajor

CourseID → CourseName

InstructorID → InstructorName

Room → Building (rooms are unique across campus)

TimeSlot, Room → CourseID, InstructorID (one section occupies one room at one time)

CourseID, InstructorID, TimeSlot → Room (a section is taught by one instructor, in one room, at one time)

3. Is it in BCNF?

No. For a table to be in BCNF, every determinant (left-hand side of an FD) must be a superkey. Here:

- Room → Building: Room alone is not a superkey (many students share the same room in the table), yet it determines Building. This is a BCNF violation.
- StudentID → StudentMajor: StudentID alone is not a superkey either — another BCNF violation.
- TimeSlot, Room → CourseID, InstructorID: {TimeSlot, Room} is not a superkey of the whole table (it doesn't include StudentID) — also a violation.

4. BCNF decomposition

Decompose step by step, each time splitting off a violating FD into its own table:

Step 1 — resolve Room → Building:

RoomInfo(Room, Building)

Remaining: CourseSchedule2(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room)

Step 2 — resolve StudentID → StudentMajor:

StudentInfo(StudentID, StudentMajor)

Remaining: CourseSchedule3(StudentID, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room)

Step 3 — resolve CourseID → CourseName:

```
CourseInfo(CourseID, CourseName)
```

```
Remaining: CourseSchedule4(StudentID, CourseID, InstructorID, InstructorName, TimeSlot, Room)
```

Step 4 — resolve InstructorID → InstructorName:

```
InstructorInfo(InstructorID, InstructorName)
```

```
Remaining: CourseSchedule5(StudentID, CourseID, InstructorID, TimeSlot, Room)
```

Step 5 — resolve TimeSlot, Room → CourseID, InstructorID (equivalently CourseID, InstructorID, TimeSlot → Room). Split into the section definition and the enrollment fact:

```
Section(CourseID, InstructorID, TimeSlot, Room)  -- key: {TimeSlot, Room} = {CourseID, InstructorID, TimeSlot}
```

```
Enrollment(StudentID, CourseID, InstructorID, TimeSlot)  -- key: {StudentID, CourseID, InstructorID, TimeSlot}
```

Final BCNF schema:

```
RoomInfo(Room, Building)
```

```
StudentInfo(StudentID, StudentMajor)
```

```
CourseInfo(CourseID, CourseName)
```

```
InstructorInfo(InstructorID, InstructorName)
```

```
Section(CourseID, InstructorID, TimeSlot, Room)
```

```
Enrollment(StudentID, CourseID, InstructorID, TimeSlot)
```

Every remaining determinant (Room, StudentID, CourseID, InstructorID, {CourseID, InstructorID, TimeSlot}, and the full key of Enrollment) is now a candidate key of its own table — BCNF is satisfied.

5. Potential loss of information

This decomposition is lossless: joining Section and Enrollment on (CourseID, InstructorID, TimeSlot) exactly reconstructs which room and building each student attended, because {CourseID, InstructorID, TimeSlot} is a key of Section (the common attributes form a superkey of one of the two decomposed relations, satisfying the lossless-join test). No spurious rows are introduced by the join.

However, the decomposition is not dependency-preserving in the strictest sense for one derived rule: the constraint "TimeSlot, Room → CourseID, InstructorID" (a room can only host one section at a time) can no longer be checked by looking at a single decomposed table alone — it now requires joining Section back together (or adding a uniqueness constraint on (TimeSlot, Room) in Section) to enforce. This is a well-known, acceptable trade-off in BCNF decompositions: BCNF guarantees losslessness but does not always guarantee dependency preservation, so an extra constraint (e.g., a UNIQUE index on (TimeSlot, Room) in Section) should be added to enforce that particular business rule.

Part 5: Design Challenge

Task 5.1: Student Clubs and Organizations

1. ER diagram — entities, attributes, relationships

Entity	Key attributes
Student	StudentID (PK), Name, Email, Major
Club	ClubID (PK), ClubName, FoundedDate, AdvisorID (FK)

Entity	Key attributes
Faculty	FacultyID (PK), Name, Department
Membership	StudentID (FK), ClubID (FK), JoinDate — composite PK
OfficerPosition	StudentID (FK), ClubID (FK), PositionTitle, TermStart, TermEnd
Event	EventID (PK), ClubID (FK), EventName, DateTime, RoomID (FK)
Attendance	EventID (FK), StudentID (FK) — composite PK, CheckInTime
Room	RoomID (PK), Building, Capacity
Expense	ExpenseID (PK), ClubID (FK), Amount, Date, Description

Relationships

- Student (M) — joins — (N) Club, resolved as Membership (M:N with attribute JoinDate)
- Faculty (1) — advises — (N) Club (each club has exactly one advisor; a faculty member can advise many clubs)
- Student (M) — holds — (N) Club as OfficerPosition (a student can hold offices in multiple clubs over time; a club has multiple officers) — modeled as its own associative entity because it needs PositionTitle/Term attributes distinct from plain membership
- Club (1) — hosts — (N) Event
- Event (1) — reserved in — (1) Room (a room reservation is a 1:1/1:N attribute link from Event to Room at a given time)
- Student (M) — attends — (N) Event, resolved as Attendance
- Club (1) — incurs — (N) Expense

2. Normalized relational schema

Student(StudentID, Name, Email, Major)

Faculty(FacultyID, Name, Department)

Club(ClubID, ClubName, FoundedDate, AdvisorID*)

Membership(StudentID*, ClubID*, JoinDate)

OfficerPosition(StudentID*, ClubID*, PositionTitle, TermStart, TermEnd)

Room(RoomID, Building, Capacity)

Event(EventID, ClubID*, EventName, DateTime, RoomID*)

Attendance(EventID*, StudentID*, CheckInTime)

Expense(ExpenseID, ClubID*, Amount, Date, Description)

(* denotes a foreign key.) Each table's non-key attributes depend on the whole key and nothing but the key, so this schema is already in 3NF/BCNF.

3. A design decision with multiple valid options

Officer positions could have been modeled two different ways:

- Option A (chosen): a separate OfficerPosition table with (StudentID, ClubID, PositionTitle, TermStart, TermEnd) as its own associative entity.
- Option B: add a nullable "Position" column directly to the Membership table.

Option A was chosen because a single student-club membership can span multiple officer terms over time (e.g., Secretary one year, President the next), each with its own start/end dates — Option B's single nullable column cannot represent that history without overwriting the previous term. A separate table also cleanly supports a membership with no officer role at all (no row in OfficerPosition) without needing NULLs scattered through Membership, keeping the design closer to 3NF and avoiding an update anomaly if a student's position changes mid-year.